SmartCampus Hybrid AI Integration

Technical Specification & Implementation Guide – v1.0 · March 2026
Audience: Development Team · School Administrators · IT Operations
Module Dependencies: Student Portal, Teacher Portal, Admin Dashboard, Communication Module, Analytics Module

---

1. Objective

Enable SmartCampus users to access AI-powered assistance while minimizing costs:

- **Students:** Use free AI web embeds (ChatGPT GPT-3.5, Perplexity basic tier).
- **Teachers/Admins:** Access automated AI features (summaries, analytics, grading assistance) via paid API integration.

This hybrid approach balances cost, accessibility, and advanced functionality.

---

2. System Architecture Overview

2.1 Hybrid Workflow

1. **Students** → click direct link or embedded widget → free AI tool opens in portal.
2. **Teachers/Admins** → input query → backend calls AI API (OpenAI GPT-3.5/4, Perplexity Pro) → response returned to portal.

Key Design Principles:

- Role-based routing
- Cost management (free vs paid usage)
- Secure API key handling
- Scalable backend architecture

Diagram: (API paths and workflow)

```
+----------------+
|   Student UI   |
+----------------+
        |
        | Direct Link / Web Embed
        v
 Free AI Service (GPT-3.5 / Perplexity)
        |
        v
   Student sees AI response

+----------------+
| Teacher/Admin  |
|      UI        |
+----------------+
        |
        | POST /api/ask-ai
        v
+----------------+
| SmartCampus    |
|  Backend       |
+----------------+
        |
        | API Call
        v
+----------------+
| AI Service API |
| (GPT-3.5/4)    |
+----------------+
        |
        v
Response → Backend → Teacher/Admin UI
```

---

3. Implementation Components

3.1 Backend Integration

- **Endpoint:** `/api/ask-ai`
- **Method:** POST
- **Input:** JSON `{ "query": "<teacher input>", "module": "<module_name>" }`
- **Output:** JSON `{ "answer": "<AI response>" }`

Python Example:

```
import openai
from flask import Flask, request, jsonify

app = Flask(__name__)
openai.api_key = "YOUR_OPENAI_API_KEY"

@app.route('/api/ask-ai', methods=['POST'])
def ask_ai():
    data = request.json
    query = data.get('query')
    module = data.get('module', 'general')

    response = openai.ChatCompletion.create(
        model="gpt-3.5-turbo",
        messages=[
            {"role": "system", "content": f"You are a SmartCampus AI
assistant for {module}."},
            {"role": "user", "content": query}
        ],
        max_tokens=300
    )
    answer = response['choices'][0]['message']['content']
    return jsonify({"answer": answer})

if __name__ == "__main__":
    app.run()
```

3.2 Student Free Embed

- **Implementation:** Iframe or portal button linking to free AI web tool.
- **Notes:** No API key required, no cost, limited features.
- **Example:**

```
<iframe src="https://chat.openai.com/" width="100%" height="600px"></iframe>
```

---

4. Role-Based Access & Workflow

Role	Access Method	API Usage	Notes
Student	Direct embed / link	Free	Limited GPT-3.5 / Perplexity basic
Teacher	Portal input → backend	Paid API	Summaries, grading, analytics
Admin	Portal input → backend	Paid API	System-wide reports, multi-module queries

- Backend automatically **detects role** and routes request accordingly.
- Users with multiple roles (Teacher+Parent) see only modules permitted for each role.

---

5. Security & Compliance

1. **API Key Management**
   - Store keys in environment variables or secure vaults.
   - Never expose API keys in client code.
2. **Access Control**
   - RBAC: Only teacher/admin roles can trigger paid API calls.
   - Audit logs for each request (user, timestamp, query type).
3. **Data Privacy**
   - Student queries sent to AI APIs must comply with Kenya Data Protection Act.
   - Avoid sending sensitive personal information unless encrypted.

---

6. Cost Management

- **Free Tier:** Students → $0 cost
- **Paid API:** Teachers/Admins → billed per token/query
  - GPT-3.5: ~$0.002 / 1K tokens
  - GPT-4: $0.03–$0.06 / 1K tokens
- **Recommendation:** Monitor usage and limit query tokens per request to manage budget.

---

7. Implementation Steps

1. **Backend Setup**
   - Install dependencies (Python: Flask + openai)
   - Configure environment variables for API keys
2. **Create API Endpoint**
   - `/api/ask-ai` → accepts POST, routes queries by role
3. **Student Free Embed**
   - Add iframe or link button in student portal
4. **Teacher/Admin Portal**
   - Input form → POST to backend endpoint
   - Display AI response dynamically
5. **Role Detection & Routing**
   - Backend reads session or user profile
   - Select free vs paid path accordingly
6. **Logging & Monitoring**
   - Track token usage and queries per role
   - Alert if monthly cost threshold exceeded
7. **Testing**
   - Local testing using Postman or curl
   - QA for role-based routing and security

8. **Deployment**
    - Deploy backend (Heroku, AWS, DigitalOcean)
    - Update portal frontends with new endpoints and embeds

---

8. Optional Enhancements

- **Caching Responses** for common queries to save API costs.
- **Predefined Templates** for automated grading summaries or analytics queries.
- **Hybrid Analytics Dashboard** showing free vs paid AI usage by role/module.

---

9. Maintenance

- Monitor **API version changes** and update endpoints.
- Audit **query logs** weekly for usage and data privacy compliance.
- Optimize **token limits** and caching to reduce costs.